

159.102 Notes

Section 6 : Pointers and Call by Reference

A look back at machine code

1RXY **Load** register R with the value XY.

e.g. 1213 means load register R2 with the number 19 (13 in hexadecimal).

3RXY **Load** register R with the number in the memory location at address XY.

e.g. 34AB means load register R4 with the contents from memory location 170 (AB).

Machine code has two different ways to load a number into a register, namely:

1RXY – this is a **direct load** where we specify the data (the actual number we are interested in)

3RXY – this is an **indirect load** where we specify the **address of the location** that holds the data

We will now explore how to load values indirectly in C++ by using the address of a variable.

Every memory location has an address

What happens when the following line of C++ is executed:

```
int a, b;
```

The system will set aside two memory locations and allocate the name a to one location and the name b to the other location. We often use the diagram:



We know from our machine code programming that every memory location has an address. The address is a number (similar to the index in an array) and if you know the address of a location, you can place a variable at that particular location. We can add more detail to the diagram above:



(Note – the addresses in this example are invented numbers – they are not actual addresses.)

We can now write a program to display the addresses of variables:

```
#include <iostream>
using namespace std;
```

```
int a, b;
```

```
int main() {
    a = 3;
    b = 5;
```

```

cout << "The value in a is " << a << endl;
cout << "The value in b is " << b << endl;
cout << "The address of a is " << &a << endl;
cout << "The address of b is " << &b << endl;
}

```

Important points about this program:

- use the ampersand symbol (&) to mean “the address of” or “a reference to”
- the addresses are displayed in hexadecimal

The “address of” operator (the ampersand) provides the address of a variable. We also have the “indirection” or “dereference” operator (the star) which provides the contents (the data or the value) at a particular address. Here is a program using both these operators:

```

#include <iostream>
using namespace std;

int a;

int main() {
    a = 21;
    cout << "The value in a is " << a << endl;
    cout << "The address of a is " << &a << endl;
    cout << "The data at the address " << &a << " is ";
    cout << *(&a) << endl;
}

```

Important points about this program:

- &x means “the address of the variable called x”
- *(&x) means “the value stored at (the address of the variable called x)”

If we have a variable, we can find it’s address (use the ampersand).

If we have an address, we can find what value is stored at that address (use the star).

A more common form of the diagram shows the addresses as indicators (**pointers**) to the memory locations. This is useful because we can avoid working with large hexadecimal numbers:



Thus *p* is a pointer to (or holds the address of) *a*, and *r* is a pointer to (or holds the address of) *b*.

Pointers

A pointer is a variable that holds a memory address as its value.

```

#include <iostream>
using namespace std;

int k;
int *ptr;

```

```

int main() {
    k = 17;
    ptr = &k;
    cout << "The value in k is " << k << endl;
    cout << "The address of k is " << ptr << endl;
    cout << "The data at the address " << ptr << " is ";
    cout << *ptr << endl;
}

```

In this program, we defined a “pointer to an integer”. You can define a pointer to any data type, e.g. `char* p;` or `float* r;` Note the use of the star (*) to define a pointer variable. When we load a value into a pointer variable, that value must be the address of a memory location. **Do not load ordinary integer values into a pointer.** For example, the line `ptr = 3;` will give an error.

The NULL pointer

There are times when we need to indicate that a pointer has not been loaded with an address.

```

#include <iostream>
using namespace std;

int *ptr;

int main() {
    ptr = NULL;
    cout << "The value of the pointer is " << ptr << endl;
    cout << "The data at the address " << ptr << " is ";
    cout << *ptr << endl;
}

```

Important points about this program:

- note that there is no ordinary variable in the program
- we load the value NULL into the pointer
- this is a BAD program – do not display the data of the NULL pointer, as it makes no sense

Indirection by using pointers

Pointers allow us to use **indirection**. This means that we can load values into variables by using the address of the variable (the pointer) just as we did in machine code. We do not have to use the variable name.

```

#include <iostream>
using namespace std;

int x;
int *ptr;

int main() {
    ptr = &x;
    *ptr = 12;
    cout << "The value is " << x << endl;
}

```

This program is interesting because the cout looks perfectly normal (we are displaying whatever value is stored in the variable called x). It displays 12 but how did 12 get loaded into x? We did not include the line `x = 12;` Instead, **we loaded 12 into x using indirection**. We set up the pointer to hold the address of x. Then used `*ptr = 12;` This instruction tells the system to find the variable whose address is stored in ptr and change the value of that variable to 12.

Definitions

Load a value **directly** – use the instruction `x = 12;`

Load a value **indirectly** – use the instruction `*ptr = 12;` having previously set ptr to an address

Programming style when using pointers

A quick note on programming style. The star can be placed anywhere between the data type and the name of the pointer. For example, any of these are possible and all mean the same thing:

```
int* ptr;
int * ptr;
int *ptr;
```

Different people say one is better than the other. In these notes we use `int *ptr;` in order to be consistent when we want to define two pointers on one line which would be:

```
int *ptr1, *ptr2;
```

Note that every pointer needs to have a star before it. This is **not** correct to define two pointers:

```
int* ptr1, ptr2;
```

This line of code would set up a pointer called ptr1 and an ordinary integer variable called ptr2.

Passing parameters to functions – Call by Value

This may seem like a rather sudden change away from our discussion on pointers. But we are actually still discussing pointers as we shall see.

Have a look at the C++ function to swap two values in the program below. It looks like it should work but run the program and you will see that **it does not swap the values** – they remain unchanged.

```
#include <iostream>
using namespace std;

void swop(int x, int y);

int main() {
    int a, b;
    a = 3;
    b = 5;
    cout << "a is " << a << " and b is " << b << endl;
    swop(a, b);
    cout << "a is " << a << " and b is " << b << endl;
}
```

```

void swop(int x, int y) {
// swop the contents of x and y
    int temp;
    temp = x;
    x = y;
    y = temp;
}

```

The reason why this doesn't work is that C++ uses a system called **call by value** to pass parameters to functions. Only the value is sent across to the function, not the actual variable. You can change the value of the parameters during the function (they act as local variables in the function) but the corresponding variables back in main are not affected. The reason for using call by value is that it allows us to pass the result of a calculation as a parameter, for example we could do this:

```
swop(a + 14 % 2, b - 3);
```

Of course, if you want to get a result from a function then use return (and do not make the function void). The problem is that return only returns one result. In the function swop for example, we want two results (the two variables that now have new values).

Solving the problem with Call by Value by using pointers

If you want a function to change a number of variables in main then you need to let the function know about the location (address) of the variables. If the function can access the variables (by using the address) then it can change the data in the variables. In other words, make the parameters pointers instead of ordinary values. The program below achieves what we want, i.e. the function now swops the values of a and b.

```

#include <iostream>
using namespace std;

void swop(int *ptr1, int *ptr2);

int main() {
    int a, b;
    a = 3;
    b = 5;
    cout << "a is " << a << " and b is " << b << endl;
    swop(&a, &b);
    cout << "a is " << a << " and b is " << b << endl;
}

void swop(int *ptr1, int *ptr2) {
// swop the contents of ptr1 and ptr2
    int temp;
    temp = *ptr1;
    *ptr1 = *ptr2;
    *ptr2 = temp;
}

```

Important points about this program:

- the parameters in the function are now pointers
- when we call the function, we pass the address of a and the address of b

- the function now works with a and b in main (even though it doesn't know the names)

Passing parameters to functions – Call by Reference

We have worked out that if we pass a reference (a pointer) to a function, the function is then able to modify the variable back in main by using the address (the pointer). We have done this by continuing to use the standard C++ parameter system (call by value) but passing pointers instead of values.

However, C++ provides us with an alternate system for parameter passing called call by reference. This system essentially uses the technique we developed above (passing pointers to the function) but provides syntax to enable programmers to use call by reference without explicitly using pointers.

```
#include <iostream>
using namespace std;

void swop(int & x, int & y);

int main() {
    int a, b;
    a = 3;
    b = 5;
    cout << "a is " << a << " and b is " << b << endl;
    swop(a, b);
    cout << "a is " << a << " and b is " << b << endl;
}

void swop(int & x, int & y) {
    // swop the contents of x and y
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

Important points about this program:

- it looks very similar to the first swapping program we wrote (except that it works now)
- note the use of ampersands to indicate “call by reference” (i.e. parameters can be changed)
- there appear to not be any pointers in the program (which is nice)

Summary of passing parameters

call by value – what we usually do in a C++ program. Values are passed to the function. These values can be changed but they do not change the corresponding values back in main – see the first version of the swop function that does not work (the values back in main remain unchanged).

call by value with pointers – if the parameters are pointers (see the second version of the swop function) then the function can change values back in main. We do not usually do this as working with pointers can get very messy.

call by reference – C++ provides a call by reference system where ampersands are used to indicate any variable that needs to be changed back in main – see the final version of the swop function. This is easy to do (no pointers) and the program now works correctly.

Final comment – if you want a function that changes values back in main, use the call by reference system to avoid using pointers. However, it is important to note that the call by reference system is actually making use of pointers (but hiding them from us).

***Note:** in this section “variables back in main” may refer to the variables in the main program but may also refer to variables in any other function that is calling the function of interest.*